

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS

Linguagem e Técnicas de Programação 2

ALUNOS:

Davi Augusto Marçal Rodrigues

Matheus Augusto de Carvalho Silva

Samuel Matias Almeida Irineu

TRABALHO PRÁTICO: MEGA MAN X MULTIPLAYER

Contagem, 2025

Sumário

Sumário	2
1 INTRODUÇÃO	3
1.1 Objetivos	3
1.2 Requisitos Funcionais	3
2 DESENVOLVIMENTO	4
2.1 Implementação	5
2.1.1 Classes Principais	6
2.1.1.1 Classe Jogo	7
2.1.1.2. Classe Entidade	7
2.1.1.3. Classe Personagem	7
2.1.1.4. Interface Inimigo	7
2.1.1.5. Classe Boss	7
2.1.1.6. Enum TipoAtaque	8
2.1.1.7. Classe GerenciadorColisoos	8
2.1.1.8. Classe NetworkManager	8
2.3 Dificuldades	8
3 RESULTADOS	10
4 CONCLUSÃO	15
5 REFERÊNCIA BIBLIOGRÁFICAS	18
Anexos	20
ANEXO A - Repositório do projeto no GitHub	20
ANEXO B - Link do vídeo de apresentação	20

1 INTRODUÇÃO

O desenvolvimento de aplicações online se tornou recorrente nos últimos anos. Os jogos evoluíram abrigando modos de *multiplayer* online ou em redes privadas que funcionam através de comunicação de rede. Esses modos permitem que mais de um usuário jogue no mesmo mundo de forma cooperativa ou competitiva (SANTOS, 2025)¹. Neste trabalho iremos adaptar um jogo *singleplayer* para adição do modo *multiplayer*.

No restante desta seção serão abordados os objetivos e os requisitos funcionais que o jogo deveria possuir. Nas demais seções, será abordado o processo de desenvolvimento do projeto, explicando as estratégias, estruturas de dados, as principais classes criadas e bibliotecas utilizadas. Além disso, apresentará as dificuldades durante a execução, os resultados obtidos, e por fim, a conclusão.

1.1 Objetivos

O objetivo geral consiste em trabalhar com conexão de rede no desenvolvimento de jogos, compreendendo processos de sincronização de aplicações online. O objetivo específico foi desenvolver uma versão simplificada do jogo Mega Man X – Super Nintendo (SNES)² que possua o modo *multiplayer* online, permitindo que mais de um jogador jogue simultaneamente no mesmo mundo. O jogo deveria ser desenvolvido utilizando a linguagem Java, a biblioteca de aplicações multimídia LibGDX e o padrão de projeto Iterator.

1.2 Requisitos Funcionais

Os requisitos funcionais do jogo, de paradigma de programação e versionamento de código estão listados abaixo:

- Controle do personagem principal
- Geração de inimigos e obstáculos
- Animação dos personagens
- Seguir as regras do jogo original: vidas, disputas e estruturas do mapa

- Progressão de fases
- Utilização de recursos multimídia
- Estruturação utilizando POO
- Uso de polimorfismo
- Implementação do padrão de projeto Iterator
- Compartilhamento via GitHub

Os requisitos que envolvem o desenvolvimento do modo *multiplayer* online, bem os requisitos estruturação do sistema são:

- Conexão online dos jogadores via rede local ou internet
- Sincronização de estados do jogo
- Identificação única de cada jogador
- Interação cooperativa entre os jogadores
- Gerenciamento de sessões do jogo:
 - O sistema deve permitir criar, entrar e sair de sessões do jogo
 - Apresentação do diagrama de atividades
- Definir a estrutura do sistema
 - Definir se o sistema é cliente/servidor ou cliente/cliente
 - Apresentação do diagrama de implantação

2 DESENVOLVIMENTO

Nesta seção apresentaremos o processo de implementação do sistema e as principais estratégias de desenvolvimento utilizados na execução das duas etapas: etapa referente a criação do jogo offline e progressão de fases; etapa de evolução do jogo, através da criação do modo multiplayer. Por fim, serão abordadas as dificuldades encontradas, como algumas delas foram solucionadas e como outras impactaram nos resultados obtidos.

2.1 Implementação

As fases implementadas foram inspiradas nas fases do Chill Penguin e do Spark Mandrill, bosses do jogo original. Não houve experiência de jogar o jogo, então para implementá-las, verificou-se a jogabilidade na gameplay postada no Youtube pelo canal GebirgeBR (GEBIRGEBR, 2021)³.

Para o início do desenvolvimento foi necessário a consulta ao vídeo *Java Game Development: #7.1 Scrolling*, do canal NovaFlux no Youtube, que permitiu compreender como implementar a lógica de *scrolling* em que a câmera do jogo acompanha a mudança de posição do personagem principal (NOVAFLUX, 2014)⁴.

Buscou-se manter a jogabilidade e funcionalidades semelhantes. Para implementação de funções dos personagens, tomamos como referência o projeto Donkey Kong em SFML e C++ (SAMUELMATIASSFR *et al.*, 2024)⁵, que possui lógicas adequadas de movimentações, animações e ataques dos personagens. Os *sprites* utilizados para animação são semelhantes ao jogo original e foram retirados dos sites (SPRITES-INC, [s.d.])⁶ e (THE SPRITERS RESOURCE, [s.d.])⁷.

Durante o desenvolvimento do jogo, diferentes estratégias foram utilizadas. O sistema foi totalmente criado utilizando POO. Visto que, além de ser um requisito funcional, o Java é uma linguagem de programação orientada a objetos. O desenvolvimento usando POO facilitou o entendimento do código, principalmente, do jogo em si.

O código foi separado e organizado por funções. Isso foi útil especialmente na classe `Jogo`, que ficou muito extensa devido a implementação das duas fases e dos métodos chamados no `NetworkManager`, responsáveis pela sincronização dos estados dos elementos do jogo entre cliente e servidor. As funções chamadas no `render()` (método principal da classe `Jogo` sobrescrita de `Game`), estão organizadas abaixo dele, seguindo a ordem exata em que são chamadas. Isso foi feito para melhorar a organização do código.

Durante o desenvolvimento da primeira parte do jogo, algumas técnicas foram utilizadas. Todos os personagens que não são o Megaman são inimigos, portanto implementam a interface `Inimigo` que possui métodos que os permitem interagir com o personagem principal e seus ataques.

Os ataques são gerenciados por *ArrayLists*. A cada ataque de algum personagem é criado um novo objeto de *Ataque* com suas especificações e armazenado em um *array* do respectivo personagem. Toda vez que um objeto de ataque encosta em algum personagem ou plataforma, ele é removido do *array*. O gerenciamento dos ataques usando *ArrayList* em vez de *Iterator* revelou-se mais simples e fácil, garantindo a fluidez no jogo.

A criação da classe *BOSS* foi outra escolha útil, pois ela serve de base para a criação dos inimigos principais do jogo, que são inimigos que se movimentam, atacam, possuem mais vida e ficam parados às vezes. Foi útil principalmente na movimentação desses bosses e na criação das ações aleatórias deles, facilitando a implementação da segunda fase.

Para fazer a comunicação cliente/servidor no jogo, foi usado a biblioteca indicada pelo *LibGDX* e disponível no *GitHub* (ESOTERICSOFTWARE, 2025)⁸, chamada *KryoNet*, amplamente usada para aplicações que envolvem comunicação de rede, útil principalmente para o desenvolvimento de jogos. Essa biblioteca permite que o computador faça conexão com uma porta *TCP* e *UDP*. Também possibilita serializar classes de uma aplicação usando *LibGDX*.

O sistema desenvolvido é cliente/servidor e foi implementado principalmente através da classe *NetworkManager* que é responsável por diversas funções dentro do código. Primeiro de tudo verifica o tipo de conexão, no caso se é cliente ou servidor. A escolha desse processo é feita por possibilitar que a comunicação servidor/cliente seja feita em uma única classe, no caso para facilitar a compatibilidade entre eles. Essa classe também é responsável por registrar e marcar a serialização das classes passadas entre o cliente e o servidor. Também, é usada para acionar as funções que permitem a troca de informações, sendo as funções para enviar as informações usadas no lado do servidor e do cliente respectivamente *sendToAllTCP()* e *sendToTCP()*. A coleta de dados é feita usando verificações, como identificando se o objeto passado é uma instância daquilo que é esperado receber.

2.1.1 Classes Principais

Neste tópico serão apresentadas as principais classes implementadas responsáveis pela execução do jogo e da comunicação entre cliente e servidor.

2.1.1.1 Classe Jogo

A classe `Jogo` é responsável por gerenciar a janela e os elementos do jogo. As principais funções exercidas pelas demais classes ocorrem nela, além disso, renderiza todos os personagens e ataques, gerenciando suas funções. Essa classe herda características da classe `Game` (definida na biblioteca `LibGDX`) que permite a manipulação de telas. Além disso, possui funções que são usadas para enviar informações dos estados dos personagens e dos ataques do jogo para a classe `NetworkManager`.

2.1.1.2. Classe Entidade

A classe `Entidade` é a classe base para as demais classes do jogo. Seus principais atributos são `Sprite`, `Textura` e variáveis *floats* para posição. Além disso, ela possui um método de animação utilizado em todas as suas filhas. Essa animação está condicionada à mudança de posição dos elementos (personagens e ataques).

2.1.1.3. Classe Personagem

A classe `Personagem` herda as características de `Entidade`, e também adiciona métodos de `mover()`, `atacar()` e `morrer()` que são sobrescritos em alguns dos demais personagens, exceto em inimigos mais simples que só se movimentam. É a classe que utiliza polimorfismo no jogo, especificamente nesses três métodos. Também, possui outros métodos: função que aplica gravidade nos personagens; e outra função responsável por animá-los quando eles estão parados.

2.1.1.4. Interface Inimigo

A interface `Inimigo` é implementada por todos os personagens que não são o Megaman. Usada na implementação da colisão entre o personagem principal e os inimigos, possibilitando que ele tome dano ao encostar em algum deles.

2.1.1.5. Classe Boss

A classe `Boss` é a classe base para os bosses do jogo. Possui métodos responsáveis pelos movimentos dos bosses, bem como determinação de suas ações. Além disso, implementa a interface `Inimigo`. Em cada boss específico que herda de `Boss` são

sobrescritas as funções principais (que utilizam polimorfismo) herdadas de `Personagem`, além da criação da função `atualizar()`, responsável pela as ações aleatórias dos bosses.

2.1.1.6. Enum TipoAtaque

No enum `TipoAtaque` todos tipos de ataques são “instanciados” diretamente e cada `Ataque` recebe um `TipoAtaque`.

2.1.1.7. Classe GerenciadorColisoes

A classe `GerenciadorColisoes` gerencia todas as colisões do jogo. Colisões entre ataques e plataformas, ataques e personagens, Megaman e inimigos e personagens e plataformas.

2.1.1.8. Classe NetworkManager

A classe `NetworkManager` gerencia a conexão entre o servidor e o inimigo, por meio dele as informações são enviadas entre o servidor e o cliente, sendo que os inimigos são sempre passados do servidor para o cliente.

2.3 Dificuldades

Durante o desenvolvimento do jogo, diversas dificuldades técnicas foram enfrentadas. Algumas impactaram significativamente os resultados.

Primeiramente, será discutido sobre as dificuldades referentes à etapa anterior, a implementação do jogo *singleplayer*. A aplicação do padrão de projeto proposto inicialmente apresentou dificuldades significativas. No início do desenvolvimento, houve uma compreensão equivocada sobre a real necessidade de implementar o padrão *Iterator* e como implementá-lo. Essa dificuldade foi superada por meio de estudos complementares em (REFACTORING GURU, 2025)⁹ e (NAYARA GARCIA, 2024)¹⁰, e com o auxílio do professor Alisson Rodrigo dos Santos, durante uma orientação realizada no dia 16/07. A partir disso, foi possível entender a importância e a forma correta de aplicar o padrão, o que possibilitou sua implementação adequada dentro do jogo.

A criação do primeiro mapa, juntamente com a configuração de suas colisões, apresentou-se como um dos principais desafios. A dificuldade estava no uso da ferramenta Tiled, cuja interface e lógica de configuração demandam conhecimento prévio. Este problema foi mitigado a partir do aprendizado adquirido nos vídeos disponíveis no YouTube: (TERMINAL ROOT, 2022)¹¹ e (GAMEFROMSCRATCH, 2015)¹². Eles auxiliaram na compreensão do funcionamento do software e na correta aplicação das colisões no mapa. A solução encontrada foi recriar o mapa do zero, com as colisões corretamente identificadas. Dessa maneira, a produção do segundo mapa tornou-se uma tarefa bem mais simples..

Também foram observados comportamentos não convencionais durante a interação/colisão entre personagens e plataformas. Ao recriar o mapa e mudar a lógica de definir a posição dos personagens após a colisão, o problema foi parcialmente superado.

A detecção de colisão entre o personagem principal e os inimigos também apresentou problemas. Ao entrar em contato com um inimigo, o personagem principal deveria receber dano somente uma vez e parar. No entanto, ele estava morrendo ao encostar. A situação foi contornada criando uma variável de tempo de invulnerabilidade que é atualizado através do delta time da própria LibGDX. Somente quando ela chega em 0, permite que o Megaman tome dano novamente. Isso resolveu o problema de forma parcial, mas em determinados momentos, quando o Megaman renasce por exemplo, há erros na lógica de dano.

A criação dos ataques também revelou-se um problema. Ao aplicar o padrão de projeto, os ataques não estavam sendo devidamente gerenciados, deixando o programa muito lento. Então optamos por utilizar *ArrayList* para gerenciar os ataques ativos de cada personagem. Toda vez que os personagens atacavam, a posição dos ataques resetava. Para consertar, a cada ataque dos personagens, um objeto *Ataque* é criado e são removidos do *array* quando encostam em outro personagem ou plataforma.

Sobre a segunda etapa de desenvolvimento do jogo, referente à construção do modo *multiplayer* online, complicações também foram encontradas. Inicialmente, houve falhas na distribuição de funções entre os integrantes do grupo e na estruturação inicial do sistema a ser desenvolvido, que acabou gerando conflitos de código posteriormente.

Além disso, houve dificuldade em decidir o tipo de conexão que seria implementado. Após decidir seguir um modelo servidor/cliente, surgiu dificuldades em entender o que deveria estar no cliente e o que deveria estar no servidor. Com o auxílio do professor Alisson

Rodrigo dos Santos, durante uma orientação realizada no dia 23/11, foi possível compreender que elementos visuais como mapas e texturas devem ser carregados no cliente, enquanto o gerenciamento dos estados dos elementos, como as posições de cada um, deveriam ser atualizados pelo servidor.

Optou-se por seguir o modelo do protótipo criado até ali, um servidor que também é jogador. Ele não controla os estados do jogo apenas, mas também joga. Essa abordagem dificultou o entendimento acerca de como o projeto está estruturado entre os integrantes do grupo, de como funciona o servidor e os clientes. Houve falhas na sincronização dos inimigos e ataques entre o servidor e os clientes, bem como dos estados gerais do jogo, como progressão de fase. Após revisão e melhor entendimento do código, foi possível realizar as sincronizações.

Por fim, houve falhas no versionamento de código. Foram realizados *commits* errados principalmente na *branch main*, o que gerou conflitos de códigos e perdas de evolução. Houve falha de comunicação entre os integrantes do grupo nesse quesito.

3 RESULTADOS

O desenvolvimento do jogo se deu em duas etapas: criação do jogo *singleplayer* com progressão de fases; e a criação do modo *multiplayer* online. A proposta visava atender a uma série de requisitos funcionais previamente estabelecidos, que abrangeram desde a estruturação da aplicação completa até aspectos visuais e sonoros. A primeira etapa foi alcançada e seus respectivos requisitos foram contemplados com sucesso. A segunda etapa e seus requisitos não foram amplamente alcançados.

Primeiramente, os resultados da primeira etapa serão discutidos. O controle do personagem principal foi implementado com sucesso. O personagem Megaman pode ser controlado utilizando o teclado, permitindo movimentações básicas como andar, pular, atacar e escalar paredes, semelhante ao jogo original.

A geração de inimigos foi implementada. O sistema conta com inimigos que surgem no cenário e possuem comportamentos básicos de movimentação e colisão. No entanto, alguns inimigos são carregados em posições bugadas nas paredes laterais. Acredita-se que

esse problema está relacionado com erros de lógica na colisão que não foram solucionados. Ainda assim, houve uma evolução em comparação com a versão anterior ao jogo, visto que, os inimigos não ficam bugados nas plataformas horizontais.

A animação dos personagens foi um dos pontos mais bem executados. O uso de sprites permitiu a criação de animações visivelmente fluidas para os personagens e os ataques. As sequências de animação foram integradas de forma mais simplificada, mas coerente com os eventos do jogo original.

No que se refere às regras do jogo tema, foi possível incorporar alguns aspectos essenciais, como o sistema de vidas e estruturas do mundo. Esses componentes funcionam em nível básico, permitindo ao jogador perder vidas ao receber dano. Contudo, ainda há erros lógicos que afetam o funcionamento contínuo dessas regras, como erros de colisões e falhas nas transições de estado do personagem.

A primeira fase que já compunha o jogo anterior foi adaptada, aumentando o número de inimigos e adaptando as ações deles no jogo. Antes não se movimentavam, agora se movimentam, seguindo o personagem. Além disso, a progressão de fases foi implementada com sucesso. A segunda fase é composta de outros inimigos e dois bosses, um no meio e outro no final do jogo, garantindo o aumento de dificuldade. Ela possui outro mapa, estruturado de forma semelhante a mapas do jogo original, mas graficamente diferente.

Quando um jogador morre ele volta para o início da fase em que está. No entanto, há um erro que não foi possível solucionar, se um jogador morre, independente se é *singleplayer* ou *multiplayer*, servidor ou cliente, os inimigos não renascem. Isso inclui inimigos simples e os bosses. Há um erro na lógica de renascer que não foi possível identificar.

Quanto ao uso de recursos multimídia, foram utilizadas imagens para sprites dos personagens, elementos do cenário, menus e interface gráfica, além de sons de ambientação. A implementação do padrão de projeto Iterator foi realizada com sucesso. Coleções de personagens e inimigos foram implementadas seguindo o padrão. Isso permitiu também o uso de polimorfismo.

Como dito anteriormente, o jogo foi totalmente estruturado utilizando POO. Os métodos e atributos de cada classe foram devidamente encapsulados conforme seu uso no sistema. O polimorfismo foi corretamente utilizado através das classes que herdam de Personagem.

O requisito de versionamento via GitHub foi parcialmente cumprido. O repositório do projeto foi mantido atualizado com *commits* regulares realizados pelos integrantes do grupo. No entanto, alguns *commits* foram realizados incorretamente, principalmente na *branch* principal (*main*), gerando conflitos de código e consequentes perdas de evolução.

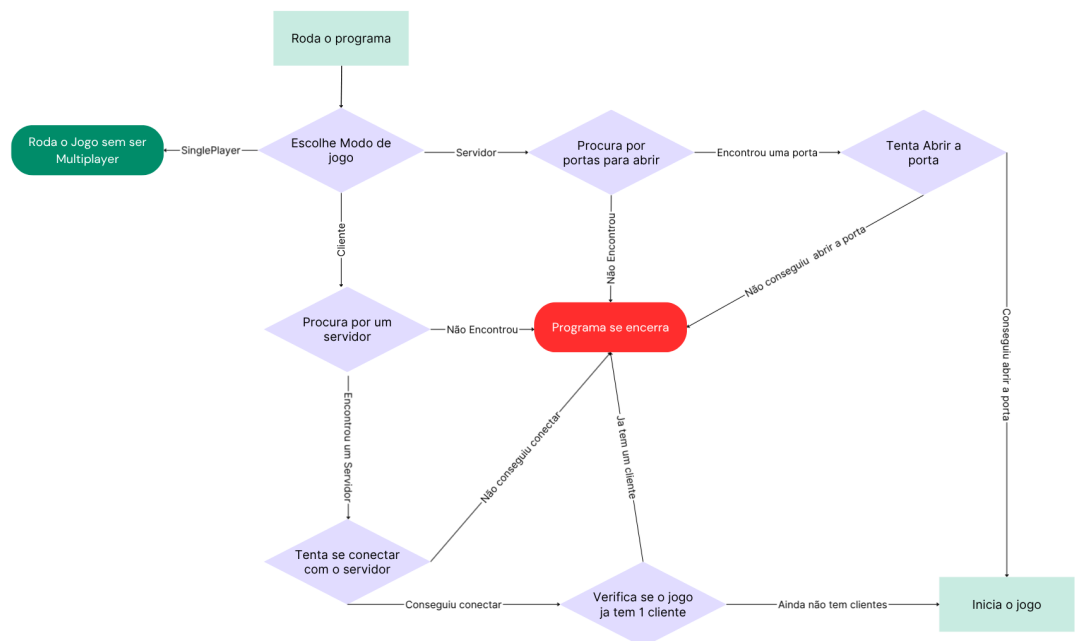
Agora os resultados referentes à segunda etapa: criação do modo *multiplayer* online serão abrangidos. A aplicação permite a conexão online de um jogador via rede local. Permite apenas um cliente condicionado a um servidor que também joga. Ambos devem ser controlados por usuários. Limitamos, mantendo a opção de dois jogadores: um servidor e outro cliente. Quando o segundo cliente tenta entrar na sala, aparece uma mensagem dizendo que a sala está cheia. Essa limitação foi feita, seguindo o esquema do jogo base, em que o Megaman batalha geralmente sozinho ou acompanhado do X, apenas um outro personagem além do principal.

A sincronização dos estados do jogo foi implementada em partes. Os inimigos simples estão devidamente sincronizados, incluindo suas posições, ataques e eventos de morte. Os ataques também são sincronizados, embora não possuam animação. Além disso, os jogadores estão bem sincronizados, tanto em posição quanto em animação. No entanto, possui um bug na sincronização dos bosses. Eles aparecem quando um dos jogadores chegam perto. Para o que chegar primeiro fica correto e permite que haja a interação competitiva, mas para o outro, não. Além disso, para o que chegar depois, a textura fica bugada, como de um inimigo simples qualquer. Isso revela um erro de lógica e comunicação que não foi possível resolver devido a complexidade do sistema.

A identificação única dos jogadores foi feita diferenciando cliente e servidor nas cores de seus sprites. O servidor é azul e o cliente é verde. O sistema permite a interação cooperativa entre eles, devido a sincronização dos inimigos no mesmo mundo.

O gerenciamento de sessões ocorre no jogo, o jogador pode conectar-se ao jogo como cliente ou servidor e caso encerre o programa também é encerrada sua participação no jogo *multiplayer*, abrindo novos lugares para uma outra conexão *multiplayer*. O diagrama de atividades da Figura 1 demonstra como funcionam as sessões do jogo.

Figura 1 - Diagrama de atividades



Fonte: Autoria própria.

Legenda: representação do gerenciamento de sessões do jogo.

A arquitetura de rede do jogo foi desenvolvida no modelo cliente/servidor, de modo a garantir uma autoridade central responsável pelas decisões críticas da simulação, como a aplicação de dano, o controle de fases e a movimentação de inimigos. Esse padrão reduz inconsistências entre as instâncias do jogo, facilita a sincronização de estado e simplifica a reconciliação de eventos durante a execução.

No sistema implementado, o servidor atua como fonte autoritativa de dados, mantendo em sua estrutura principal, representada pelo componente Jogo, o estado global e as instâncias relevantes da simulação, como `EnemyPosition`, `PlayerPosition`, `PenguinState`, `SparkState`, `VileState`, `ConnectionRejected`, `PosicaoTiro`, `GerenciadorFases` e `VerificaGanhar`. Essas instâncias correspondem aos objetos e eventos do jogo que precisam ser compartilhados com os clientes, sendo gerenciadas internamente pelo servidor.

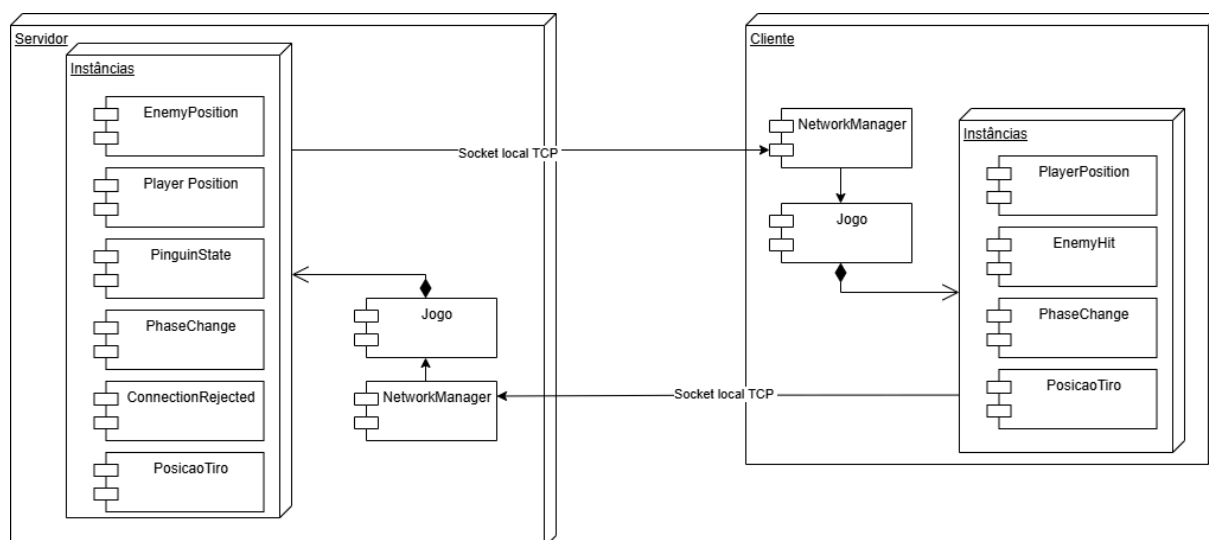
O cliente, por sua vez, também possui um componente, contendo suas próprias instâncias locais — de `PlayerPosition`, `EnemyHit`, `PosicaoTiro`,

`GerenciadorFases` e `VerificaGanhar` —, utilizadas para exibir e atualizar o estado recebido do servidor. A comunicação entre cliente e servidor é realizada por meio de sockets TCP locais, gerenciados pelo módulo `NetworkManager`, que é responsável pelo envio e recebimento de mensagens estruturadas em objetos simples (DTOs).

Durante a execução, uma instância pode atuar como servidor, abrindo portas até estabelecer a conexão, ou como cliente, tentando se conectar ao servidor. O cliente envia informações sobre sua posição e ações, como tiros ou acertos, enquanto o servidor processa esses eventos de forma autoritativa, atualiza o estado global e devolve ao cliente uma versão sincronizada e resumida, além de repassar mensagens relevantes entre os participantes.

A implementação também considera aspectos de robustez e estabilidade. A serialização é restrita a DTOs simples, evitando o envio de elementos pesados como texturas e sons. Os *buffers* do Kryo foram ampliados para prevenir estouros de memória (*overflows*), e as operações que interagem com a API gráfica são executadas na thread principal por meio de `Gdx.app.postRunnable`. Há ainda tratamento para falhas de conexão, com tentativas automáticas em portas consecutivas e rejeição de novas conexões quando a sala já está cheia. Por fim, foram implementados métodos de limpeza e descarte (`dispose()`) para encerrar corretamente o cliente e o servidor, priorizando a estabilidade da sessão e a integridade do estado do jogo. O diagrama de implantação da Figura 2 apresenta como o sistema foi desenvolvido.

Figura 2 - Diagrama de implantação



Fonte: Autoria própria.

Legenda: representação de como o sistema cliente/servidor foi estabelecido.

4 CONCLUSÃO

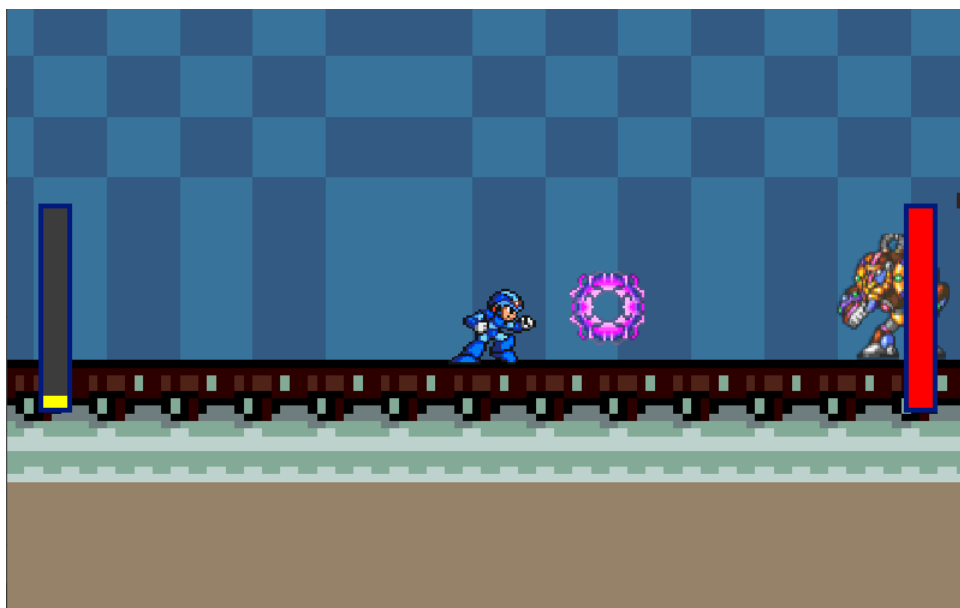
Conclui-se que o projeto não atingiu todos os objetivos de forma satisfatória aos criadores, mas de forma que entrega os principais requisitos. A primeira etapa revelou resultados bem satisfatórios e condizentes com os requisitos. O objetivo geral de trabalhar com conexão de rede no desenvolvimento de jogos, criando uma sincronização simples de aplicações online. No entanto, é possível identificar a existência de alguns *bugs* que ao tentarmos resolver desencadeou uma série de outros, causados pela forma em que a lógica do jogo foi trabalhada.

Além disso, é possível concluir que houve uma falha na estruturação do sistema cliente/servidor. A abordagem utilizada de servidor que também joga não se mostrou a mais adequada para o grupo. Visto que, provocou certa confusão acerca das suas funções e da forma de gerenciá-las no cliente e no servidor.

Embora o jogo possui alguns bugs e erros de sincronização, foi possível implementar uma versão funcional do jogo com um modo de gameplay *multiplayer* que estabelece uma conexão estável e é capaz de demonstrar e atualizar os movimentos de quase todos os personagens de forma simultânea entre os jogadores, com exceção dos bosses.

A Figura 3 apresenta o enfrentamento do *boss* na segunda fase no modo *singleplayer*. A Figura 4 mostra a segunda tela do jogo que permite escolher entrar no modo *singleplayer*, servidor ou cliente. Por fim, a Figura 5 mostra os clientes no mesmo mundo, com sincronização de inimigos.

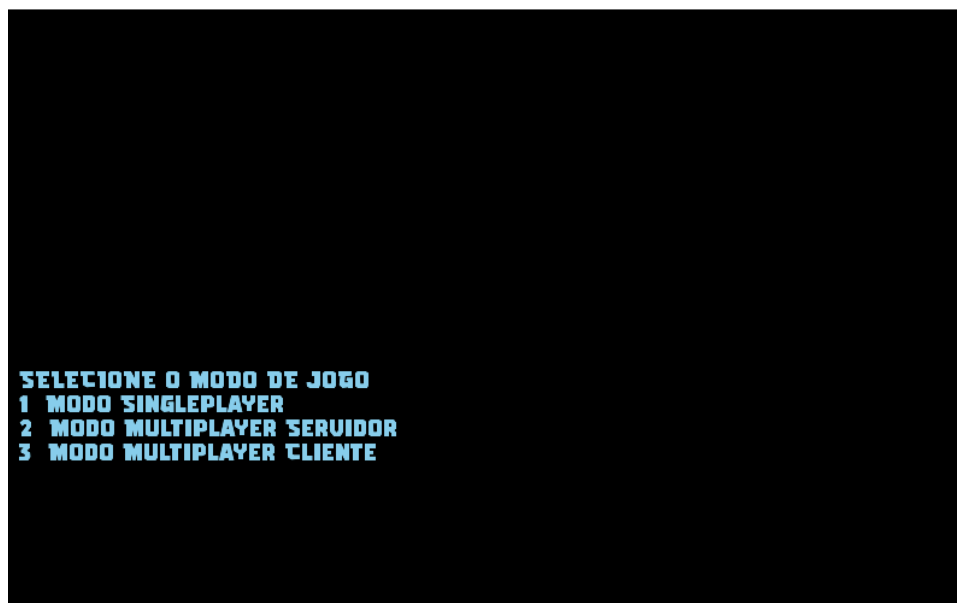
Figura 3 - Imagem da segunda fase do jogo



Fonte: Autoria própria.

Legenda: Megaman enfrentando o boss da segunda fase, Spark Mandrill.

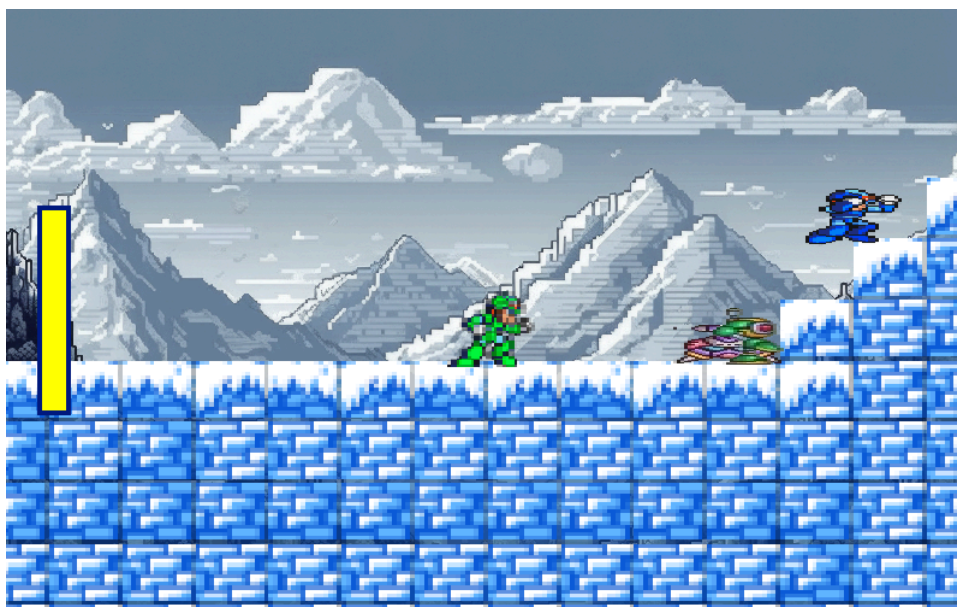
Figura 4 - Tela inicial do jogo



Fonte: Autoria própria.

Legenda: tela inicial do jogo que permite o usuário escolher o modo de jogo.

Figura 5 - Jogadores e inimigos sincronizados



Fonte: Autoria própria.

Legenda: representação dos clientes e os inimigos sincronizados no mesmo mundo.

5 REFERÊNCIA BIBLIOGRÁFICAS

- ¹ SANTOS, Alisson Rodrigo dos. **Network Game**. CEFET-MG, curso Técnico em Informática – 3º ano, Trabalho Prático 3 da disciplina LP2/LLP2. 2025. Disponível em: <http://gpjecc.blogspot.com.br/>. Acesso em: 3 nov. 2025.
- ² RETROGAMES.CZ. Mega Man X – Super Nintendo (SNES). [S.l.]: RetroGames.cz, [2025?]. Disponível em: https://www.retrogames.cz/play_865-SNES.php. Acesso em: 5 jul. 2025.
- ³ GEBIRGEBR. *Megaman X – ZERADO 100% – Hadouken, Corações e todos os itens* [vídeo]. YouTube, 2021. Disponível em: https://www.youtube.com/watch?v=y_ucHpCNO4E&t=1144s. Acesso em: 24 jun. 2025.
- ⁴ NOVAFLUX. *Java Game Development: #7.1 Scrolling* [vídeo]. YouTube, 2014. Disponível em: <https://www.youtube.com/watch?v=N-Sx-RcYvQc>. Acesso em: 2 nov. 2025.
- ⁵ SAMUELMATIASSFR. DonkeyKongSFML [repositório GitHub], 2024. Disponível em: <https://github.com/SamuelMatiasSfr/DonkeyKongSFML>. Acesso em: 5 jul. 2025.
- ⁶ SPRITES INC. Mega Man X – Sprites. [S.l.]: Sprites Inc., [s.d.]. Disponível em: <https://www.sprites-inc.co.uk/sprite.php?local=/X/>. Acesso em: 5 jul. 2025.
- ⁷ THE SPRITERS RESOURCE. Mega Man X (SNES) – Sprites. [s.d.]: The Spriters Resource. Disponível em: <https://www.spritters-resource.com/snes/mmx/>. Acesso em: 5 jul. 2025.
- ⁸ ESOTERICSOFTWARE. *Kryo: Java binary serialization and cloning: fast, efficient, automatic*. GitHub repository. Disponível em: <https://github.com/EsotericSoftware/kryo>. Acesso em: 24 jun. 2025.
- ⁹ REFACTORING GURU. Padrão do projeto Iterator. Disponível em: <https://refactoring.guru/pt-br/design-patterns/iterator>. Acesso em: 24 jun. 2025.
- ¹⁰ NAYARA GARCIA. *Padrão de Projeto Iterator – Implementação Java*. [vídeo no YouTube]. YouTube, 2024. Disponível em: https://www.youtube.com/watch?v=Xd_8hVDA61U. Acesso em: 24 jun. 2025.

¹¹ TERMINAL ROOT. *Como Criar Mapas para Games no TiledMap*. [vídeo no YouTube]. YouTube, 2022. Disponível em: https://www.youtube.com/watch?v=Xd_8hVDa61U. Acesso em: 24 jun. 2025.

¹² GAMEFROMSCRATCH. *Tutorial do Editor de Mapas em Mosaico - Parte Três: AutoMap*. [vídeo no YouTube]. YouTube, 2015. Disponível em: https://www.youtube.com/watch?v=A_A6rz7cvG4&list=PLu4oc9P-ABcOXNOyoAvnMyUwn_kkiVA5B. Acesso em: 24 jun. 2025.

Anexos

ANEXO A - Repositório do projeto no GitHub

O código-fonte completo do projeto está disponível no GitHub.

Link para o repositório: <https://github.com/Grupo-8-2025/TrabalhoPratico3>

ANEXO B - Link do vídeo de apresentação

O vídeo da apresentação do jogo está disponível no Youtube.

Link para o vídeo: <https://youtu.be/KpQt37bqn58>